

AI-Powered Placement Management Platform (PlaceAI)

Executive Summary

Campus placements represent one of the most important milestones in a student's academic journey. However, placement processes in many institutions are still dependent on spreadsheets, manual verification, repetitive communication, and disconnected systems.

Students often struggle to identify opportunities that match their skills. Recruiters spend considerable time reviewing hundreds of applications, while placement officers manage large volumes of data with limited automation.

PlaceAI is an AI-powered Placement Management Platform designed to transform the placement ecosystem by providing intelligent job recommendations, automated candidate screening, skill-gap analysis, and centralized placement operations.

The platform combines Artificial Intelligence, secure system design, cloud-native architecture, and modern development practices to create a scalable and efficient placement management solution.

Problem Statement

Current placement systems face several challenges:

Student Challenges

- Applying to irrelevant job opportunities.
- Lack of visibility into eligibility criteria.
- Limited feedback on resume quality.
- Difficulty identifying missing skills.

Recruiter Challenges

- High volume of applications.
- Time-consuming resume screening.
- Difficulty identifying top candidates quickly.

Placement Team Challenges

- Manual management of placement drives.
- Repetitive communication tasks.
- Difficulty generating accurate reports.
- Lack of centralized analytics.

These challenges result in delayed hiring decisions, lower placement efficiency, and increased administrative workload.

Proposed Solution

PlaceAI addresses these challenges through:

AI-Powered Services

- Resume Parsing
- Smart Job Recommendations

- Candidate Ranking
- Skill Gap Analysis
- Placement Analytics

Placement Management Services

- Student Profile Management
- Placement Drive Management
- Application Tracking
- Interview Scheduling
- Notification Management

Enterprise Services

- Secure Authentication
- Role-Based Access Control
- Audit Logging
- Monitoring and Reporting

Team Deliverables:

1. Stakeholder Analysis

Students

Students are the primary beneficiaries of the platform.

Goals

- Find suitable opportunities.
- Improve placement readiness.
- Track application progress.

How PlaceAI Helps

- Personalized job recommendations.
- Resume evaluation and improvement suggestions.
- Skill development guidance.

Recruiters

Recruiters need efficient access to qualified candidates.

Goals

- Reduce screening time.
- Improve candidate quality.
- Streamline hiring workflows.

How PlaceAI Helps

- AI-powered candidate ranking.
- Advanced candidate filtering.
- Automated interview scheduling.

Placement Officers

Placement officers manage the overall placement process.

Goals

- Reduce manual effort.
- Improve coordination.
- Generate reports quickly.

How PlaceAI Helps

- Centralized dashboards.
- Automated notifications.
- Real-time analytics.

College Administration

Goals

- Monitor placement performance.
- Improve institutional outcomes.

How PlaceAI Helps

- Department-wise analytics.
- Placement trend reports.
- Recruiter engagement insights.

Stakeholder	Goals	Challenges
Students	Get placed	Lack of guidance
Recruiters	Hire talent	Too many resumes
Placement Officers	Manage drives	Manual tracking
Faculty	Guide students	Limited visibility
Administration	Analytics	Data fragmentation

2. User Stories

Student

As a student, I want personalized job recommendations so that I can focus on opportunities relevant to my profile.

As a student, I want resume analysis so that I can improve my chances of being shortlisted.

As a student, I want to track my applications so that I know my current recruitment status.

Recruiter

As a recruiter, I want AI-ranked candidates so that I can identify suitable applicants quickly.

As a recruiter, I want advanced filtering so that I can search candidates efficiently.

Placement Officer

As a placement officer, I want placement analytics and reports so that I can monitor placement performance.

As a placement officer, I want automated notifications so that students do not miss important deadlines.

3. Product Requirements Document (PRD)

Functional Requirements

Authentication Module

- Student Login
- Recruiter Login
- Placement Officer Login
- JWT Authentication
- Refresh Tokens

Student Module

- Profile Creation
- Resume Upload
- Skill Management
- Job Applications
- Application Tracking

Recruiter Module

- Company Registration
- Job Posting
- Candidate Search
- Candidate Ranking
- Interview Scheduling

Placement Officer Module

- Placement Drive Creation
- Eligibility Management

- Analytics Dashboard
- Report Generation

AI Module

- Resume Parsing
- Skill Extraction
- Candidate Matching
- Recommendation Engine
- Skill Gap Detection

Technology Stack

Layer	Technology
Frontend	React.js, TypeScript, Tailwind CSS
Backend	Node.js, Express.js
ORM	Prisma ORM
Database	MongoDB (MVP)
AI	Python, FastAPI, Scikit-Learn
DevOps	Docker, NGINX, GitHub Actions
Cloud	AWS

Prisma acts as an abstraction layer between the application and database.

Benefits:

- Faster development.
- Type-safe database queries.
- Easier maintenance.
- Future database flexibility.

Current Architecture:

React → Node.js → Prisma → MongoDB

Future Architecture:

React → Node.js → Prisma → PostgreSQL

This approach allows the platform to start quickly with MongoDB while remaining adaptable for future enterprise-scale requirements.

Security Architecture

Security is treated as a core requirement rather than an afterthought.

Authentication

- JWT Access Tokens
- Refresh Tokens
- Session Expiry

Password Security

Passwords are encrypted using BCrypt hashing before storage.

Passwords are never stored in plain text.

Role-Based Access Control

Student Access

Allowed:

- View own profile
- Apply for jobs

Restricted:

- Recruiter dashboards
- Administrative controls

Recruiter Access

Allowed:

- Manage job postings
- Review candidates

Restricted:

- Administrative modules

Admin Access

Allowed:

- Platform management
- User management
- Reporting

API Security

No API endpoints are publicly accessible.

Every protected endpoint requires:

- JWT Validation
- Authorization Middleware
- Role Verification

Unauthenticated requests receive:

401 Unauthorized

Unauthorized role access receives:

403 Forbidden

NGINX Security Layer

NGINX serves as the gateway to the application.

Responsibilities:

- Reverse Proxy
- SSL Termination
- HTTPS Enforcement
- Rate Limiting
- Request Filtering

Benefits:

- Backend services remain hidden.
- Reduced attack surface.
- Improved performance.

Security Headers

Implemented through NGINX:

- Content Security Policy
- Strict Transport Security
- X-Frame-Options
- X-Content-Type-Options

Database Security

- Principle of Least Privilege
- Encrypted Sensitive Fields
- Daily Automated Backups
- Disaster Recovery Strategy

4. Feature Prioritization (MoSCoW)

Must Have	Should Have	Could Have
Authentication	Resume Scoring	AI Assistant
Applications	Skill Gap Analysis	Mock Interviews
Recruiter Dashboard	Interview Scheduling	Learning Paths

5. Wireframes

Login Page

Email → Password → Login/Register

Student Dashboard

Profile Completion, Resume Score, Recommended Jobs, Applications

Recruiter Dashboard

Jobs Posted, Applications Received, Top Candidates

Placement Officer Dashboard

Placement Statistics, Upcoming Drives, Reports

Evaluation Parameters

1. Product Ownership

As the Product Owner, the focus was on identifying the real challenges faced by students, recruiters, and placement officers and converting those challenges into a practical solution.

Key Contributions

- Defined the product vision and objectives.
- Identified stakeholder pain points through analysis.
- Prioritized features using the MoSCoW framework.
- Focused on delivering an MVP that provides maximum business value.
- Ensured security, scalability, and usability were considered from the initial design phase.

Product Decisions

- AI-powered job recommendations to improve placement success.
- Candidate ranking to reduce recruiter effort.
- Centralized dashboards to eliminate manual placement tracking.
- Prisma ORM to support future database flexibility.

2. Requirement Analysis

The project requirements were gathered by understanding the needs of all stakeholders and translating them into business, functional, and non-functional requirements.

Business Requirements

- Improve placement success rate.
- Reduce recruiter screening effort.
- Automate placement operations.
- Enhance student placement experience.

Functional Requirements

- User Authentication
- Resume Upload
- Job Recommendation Engine
- Candidate Ranking
- Application Tracking
- Placement Analytics Dashboard

Non-Functional Requirements

- Secure Authentication & Authorization
- Response Time < 2 seconds
- 99.9% System Availability
- Scalability for 10,000+ Users

3. System Thinking

The solution was designed by considering how different components interact within the overall ecosystem.

System Design Approach

- Modular architecture for maintainability.
- Separation of frontend, backend, AI services, and database.

- Secure API communication.
- Scalable cloud-ready deployment.

Key Considerations

- Security through JWT, RBAC, and NGINX.
- Future database migration support using Prisma ORM.
- Monitoring and logging for operational visibility.
- High availability and fault tolerance.

End-to-End Flow

Student → Frontend → Backend → Database → AI Engine → Recommendations

This demonstrates a holistic understanding of how business processes and technical systems work together.

4. Team Coordination

Successful product delivery requires collaboration across multiple roles.

Team Structure

Role	Responsibility
Product Owner	Product Vision & Prioritization
Business Analyst	Requirements Gathering
UI/UX Designer	Wireframes & User Experience
Frontend Developer	User Interface Development
Backend Developer	APIs & Security

Role	Responsibility
AI Engineer	Recommendation Engine
DevOps Engineer	Deployment & Monitoring

Collaboration Approach

- Daily stand-up meetings
- Sprint planning sessions
- Weekly review meetings
- GitHub-based task tracking
- Code reviews and pull requests

This ensured alignment between business goals and technical implementation.

5. Leadership Potential

Leadership was demonstrated through strategic decision-making, risk management, and long-term planning.

Leadership Decisions

- Adopted a security-first approach instead of adding security later.
- Selected Prisma ORM to avoid database vendor lock-in.
- Prioritized MVP features before advanced enhancements.
- Designed a scalable architecture capable of supporting future institutional growth.